Edge Detection Using OpenCV

Name: Shubham Kumar Singh

Reg No: RA2211031010018

Course: Computer Vision

Tool: Google Colab (Python, OpenCV, Matplotlib)

Introduction

Edge detection is a fundamental operation in image processing and computer vision. Edges represent boundaries between objects or regions in an image where pixel intensity changes sharply. Identifying these edges helps in understanding shapes, textures, and object structures within an image. Edge detection plays a vital role in tasks like object detection, segmentation, feature extraction, and pattern recognition.

In this experiment, we apply and compare three popular edge detection techniques — Sobel, Laplacian, and Canny — using Python and OpenCV. Each technique offers a unique approach to detecting edges and exhibits different levels of noise sensitivity, accuracy, and computational complexity.

Objectives

• Understand the mathematical concepts behind different edge detection algorithms.

• Implement Sobel, Laplacian, and Canny edge detectors using Python and OpenCV.

• Compare the performance and output of each edge detection method.

• Analyze the robustness, precision, and noise sensitivity of each algorithm.

• Visualize, interpret, and save edge maps for further analysis.

Theory / Methodology

1. Sobel Operator

The Sobel operator is a gradient-based edge detector that identifies edges by calculating the first derivative of image intensity. It uses convolution kernels in both horizontal (X) and vertical (Y) directions to emphasize regions with high intensity variation.

Mathematically:

Gx = [[-1 0 1], [-2 0 2], [-1 0 1]]

Gy = [[-1 -2 -1], [0 0 0], [1 2 1]]

Gradient Magnitude = sqrt($Gx^2 + Gy^2$)

Pros:

- Simple and efficient

- Detects horizontal and vertical edges

Cons:

- Sensitive to noise

- Produces thick edges

Flowchart:

Grayscale Image → Sobel X & Y → Gradient Magnitude → Edge Map

2. Laplacian Operator

The Laplacian operator computes the second derivative of the image intensity. It highlights regions where the rate of change in brightness is high, detecting edges and fine details in all directions simultaneously.

Mathematical representation:

$\nabla^2 I = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$

Pros:

- Direction-independent (isotropic)

- Captures fine details

Cons:

- Extremely sensitive to noise

- Often requires Gaussian smoothing

Flowchart:

Grayscale Image → Laplacian Filter → Edge Map

3. Canny Edge Detector

The Canny edge detector is a multi-stage, robust algorithm designed to detect edges accurately while minimizing noise. It involves several steps for optimal edge localization.

Steps:

1. Noise Reduction: Apply Gaussian Blur

2. Gradient Computation: Identify edges using intensity gradients

3. Non-Maximum Suppression: Thin edges to keep local maxima only

4. Hysteresis Thresholding: Link strong and weak edges to form continuous contours

Pros:

- Produces clean, thin, and continuous edges

- Robust to noise

Cons:

- Computationally more intensive

Flowchart:

Grayscale Image → Gaussian Blur → Gradient Computation → Non-Max Suppression → Double Threshold → Edge Map

Implementation

The implementation is carried out in Google Colab using Python and OpenCV. The workflow includes uploading the image, resizing and converting it to grayscale, applying the three edge detection techniques, and visualizing/saving the results. Matplotlib was used for displaying comparison plots.

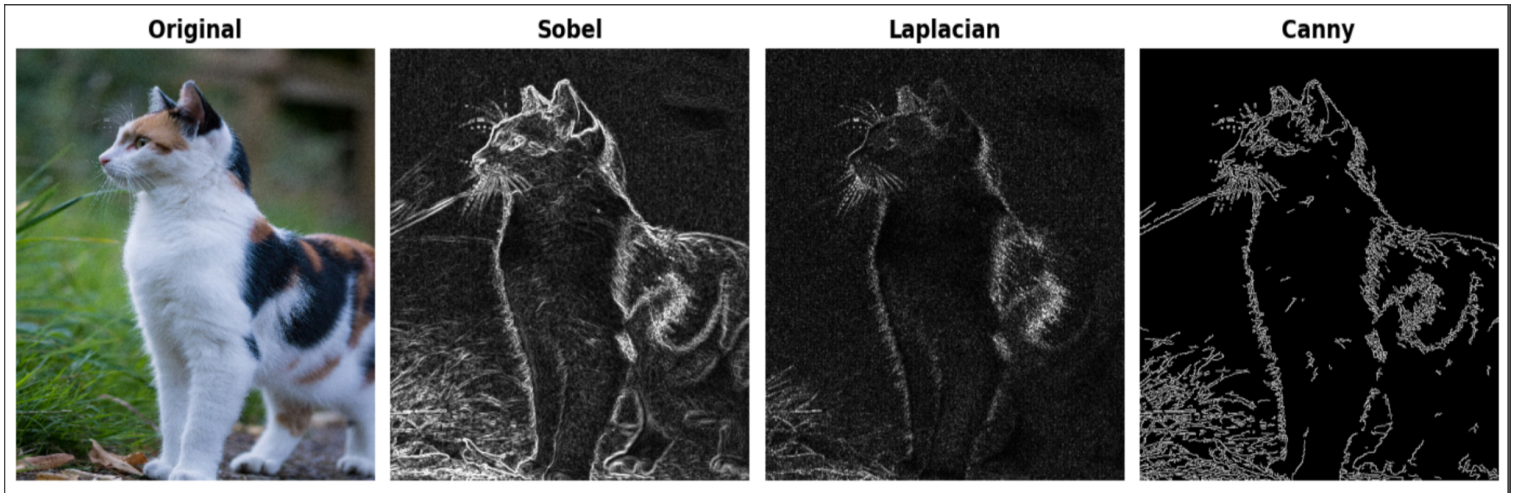
CODE:

```
import os
import cv2
import matplotlib.pyplot as plt
from PIL import Image
try:
    from google.colab import files
    COLAB = True
except ImportError:
    COLAB = False
def upload_image():
    """Upload a single image in Colab (.jpg or .png)."""
    if not COLAB:
        raise EnvironmentError("This function works only in Google Colab!")
    print("📤 Please upload your image (.jpg or .png):")
    uploaded = files.upload()

    for filename in uploaded.keys():
        filepath = os.path.join("/content", filename)
        with open(filepath, 'wb') as f:
            f.write(uploaded[filename])
        print(f"✅ Uploaded image: {filepath}")
        return filepath
uploaded_image_path = upload_image()
img_color = cv2.imread(uploaded_image_path, cv2.IMREAD_COLOR)
img_color = cv2.resize(img_color, (512, 512))
```

```
img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)

edges_sobel_x = cv2.Sobel(img_gray, cv2.CV_64F, 1, 0, ksize=3)

edges_sobel_y = cv2.Sobel(img_gray, cv2.CV_64F, 0, 1, ksize=3)

edges_sobel = cv2.magnitude(edges_sobel_x, edges_sobel_y)

edges_sobel = cv2.convertScaleAbs(edges_sobel)

edges_laplacian = cv2.Laplacian(img_gray, cv2.CV_64F)

edges_laplacian = cv2.convertScaleAbs(edges_laplacian)

edges_canny = cv2.Canny(img_gray, 100, 200)

titles = ['Original', 'Sobel', 'Laplacian', 'Canny']

images = [cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB), edges_sobel, edges_laplacian,
edges_canny]

plt.figure(figsize=(14,6))

for i in range(4):

    plt.subplot(1,4,i+1)

    if i == 0:

        plt.imshow(images[i])

    else:

        plt.imshow(images[i], cmap='gray')

    plt.title(titles[i], fontsize=14, fontweight='bold')

    plt.axis('off')

plt.tight_layout()

output_dir = "./edge_outputs"

os.makedirs(output_dir, exist_ok=True)

comparison_path = os.path.join(output_dir, "edge_comparison.png")

plt.savefig(comparison_path, bbox_inches='tight')

plt.show()

cv2.imwrite(os.path.join(output_dir, "original.png"), img_color)

cv2.imwrite(os.path.join(output_dir, "sobel_edges.png"), edges_sobel)

cv2.imwrite(os.path.join(output_dir, "laplacian_edges.png"), edges_laplacian)

cv2.imwrite(os.path.join(output_dir, "canny_edges.png"), edges_canny)

print(f"✅ Edge maps saved to: {os.path.abspath(output_dir)}")

print(f"🖼️ Combined comparison figure saved to: {comparison_path}")
```

Results

The resulting edge maps from each algorithm are shown below. Each technique emphasizes different structural details depending on how gradients and thresholds are computed.



Observations and Analysis

1. Sobel Operator:

   - Captures strong edges along major gradients.

   - Some noise and thick edges observed.

   - Suitable for simple edge extraction tasks.

2. Laplacian Operator:

   - Very sensitive to small intensity variations.

   - Highlights textures and fine details.

   - Noisy without preprocessing like Gaussian blur.

3. Canny Edge Detector:

   - Produces thin, continuous, and accurate edges.

   - Effectively filters noise.

   - Ideal for precise object boundary detection and segmentation.

In comparison, Canny provides the best trade-off between noise resistance and edge clarity, making it a preferred choice for practical computer vision applications.

Conclusion

Edge detection is an essential step in image analysis. Among methods, Sobel is simple and effective for gradients, Laplacian captures fine details but is noise-sensitive, and Canny provides the most accurate and clean edges. While Canny is ideal for precise outlines, Sobel and Laplacian are more practical for real-time or less complex systems, highlighting the trade-off between accuracy, noise handling, and efficiency.